Indice

1 AVL Trees **2**
1.1 Balanced Tree 3
1.2 AVL Tree definition 4
1.3 AVL Tree Rotations 5
1.3.1 First & Second Case 5
1.3.2 Third & Fourth Case 5
1.4 Performance 6

Maps

Appunti di Programmazione

26 marzo 2026

1 AVL Trees

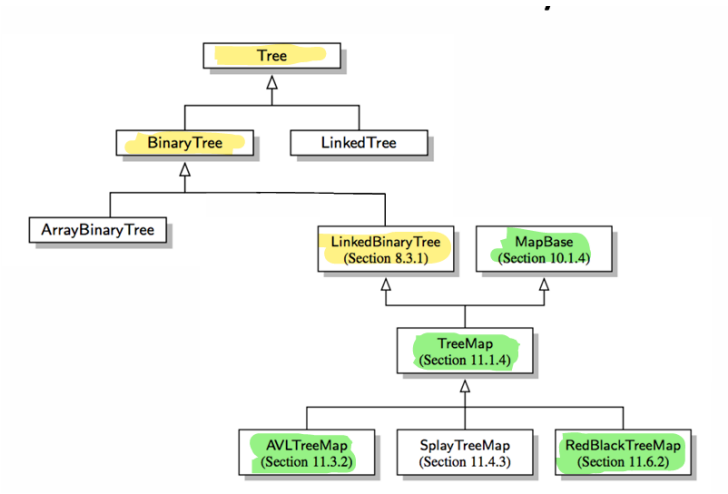

Figura 1: Gneral hierarchy

1.1 Balanced Tree

Si suppone che il nodo foglia abbia altezza pari a 1.

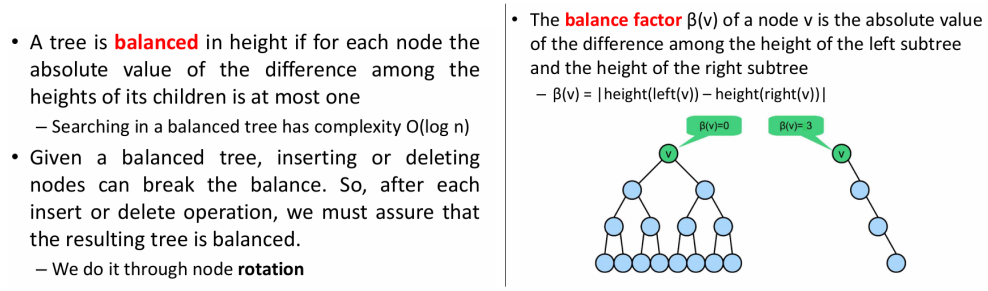

Figura 2: Balanced Tree Definition

1.2 AVL Tree definition

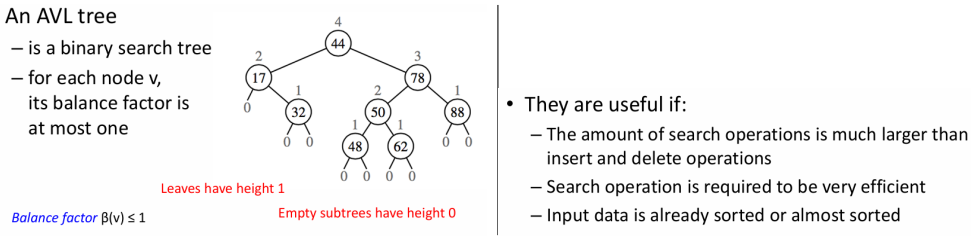

Figura 3: AVL Tree Definition

1.3 AVL Tree Rotations

Per garantire che l'AVL tree rimanga bilanciato dopo inserimenti e cancellazioni, c'è bisogno di *ruotare* i nodi.

There are four possible cases
- Two cases change the position of 2 nodes (**single rotation**)
- Two cases change the position of 3 nodes (**double rotation**)

Figura 4: AVL Tree Rotations Cases

1.3.1 First & Second Case

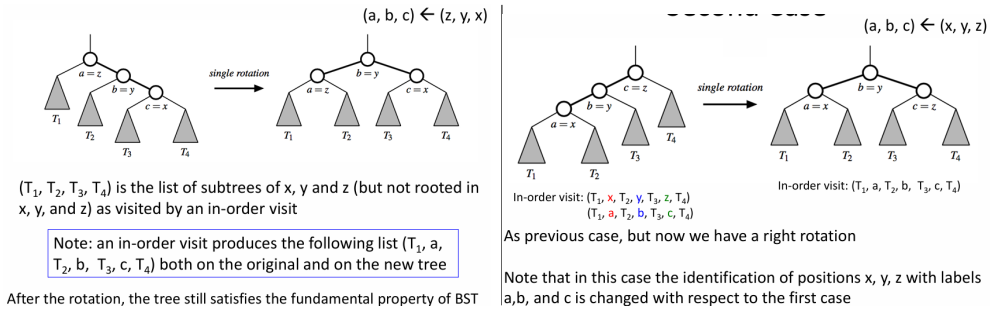

Figura 5: First & Second Case

1.3.2 Third & Fourth Case

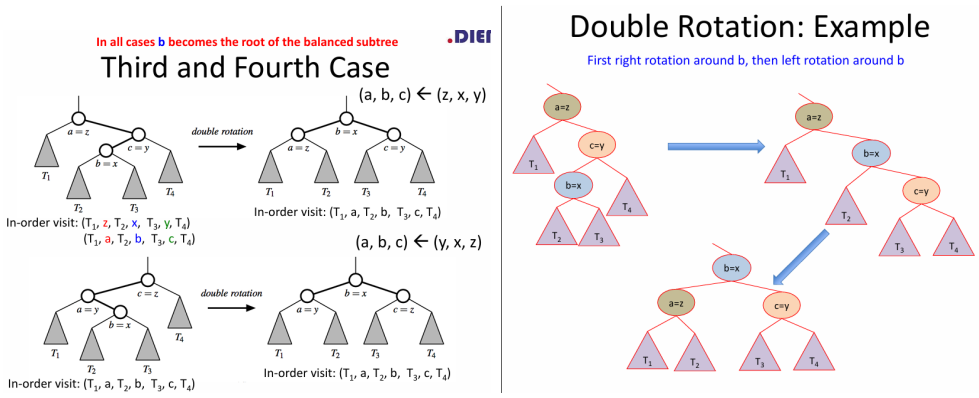

Figura 6: Third & Fourth Case

1.4 Performance

Complexity of the operations

- Recall that the height of an AVL tree with n nodes is O(log n)

Operation	Running Time
k in T	$O(\log n)$
T[k] = v	$O(\log n)$
T.delete(p), del T[k]	$O(\log n)$
T.find_position(k)	$O(\log n)$
T.first(), T.last(), T.find_min(), T.find_max()	$O(\log n)$
T.before(p), T.after(p)	$O(\log n)$
T.find_lt(k), T.find_le(k), T.find_gt(k), T.find_ge(k)	$O(\log n)$
T.find_range(start, stop)	$O(s + \log n)$
iter(T), reversed(T)	$O(n)$

AVL

	Average	Worst Case
Space	$O(n)$	$O(n)$
Search	$O(\log n)$	$O(\log n)$
Insert	$O(\log n)$	$O(\log n)$
Delete	$O(\log n)$	$O(\log n)$

BST

	Average	Worst Case
Space	$O(n)$	$O(n)$
Search	$O(\log n)$	$O(n)$
Insert	$O(\log n)$	$O(n)$
Delete	$O(\log n)$	$O(n)$

In the worst case, the height of an **AVL** tree is ~1.44 log(n),
(the height of a **Red Black** tree is 2 log(n))

Figura 7: AVL Tree Performance